

Perturbational Complexity Index (PCI) Calculator

Supplementary Material

Berke Isler

mehmetberkeisler@gmail.com

February 2026

Abstract

This document provides supplementary material for the Perturbational Complexity Index (PCI) Calculator software. It describes the theoretical background behind PCI, the complete mathematical formulation of the algorithm as proposed by Casali et al. (2013), the implementation details of our Python-based processing pipeline, the corrections and improvements applied during code review, and a formal description of the *istisna* exception-handling mode for mislabelled trigger markers. A TikZ-rendered flowchart of the full algorithm is provided along with pseudocode for all core routines.

Contents

1	Introduction	2
2	Mathematical Formulation	2
2.1	Source Entropy	2
2.2	Lempel–Ziv Complexity (LZ76)	2
2.3	PCI Normalisation	3
2.4	Temporal Variant PCI^T	3
2.5	Temporal Evolution $PCI(t)$	3
3	Bootstrap Significance Testing	3
3.1	Procedure	3
3.2	Correlated vs. Independent Resampling	4
4	Source Sorting (Optimal Ordination)	4
4.1	Deterministic Tie-Breaking	4
5	Preprocessing Pipeline	4
5.1	CSD vs. Inverse Solution	5
6	Istisna Mode: Exception Handling for Mislabelled Triggers	5
6.1	Motivation	5
6.2	Detection Algorithm	5
6.3	User Interaction	5
7	Quality Assessment	6
8	Clinical Interpretation	6
9	Algorithm Flowchart	7
10	Corrections and Improvements	8
10.1	Critical Fixes	8
10.2	Moderate Fixes	8

11 Software Architecture	8
12 Test Suite	8
13 Future Work	9

1 Introduction

The Perturbational Complexity Index (PCI) is an electrophysiological measure of brain complexity proposed by Casali et al. [1] as a theory-grounded indicator of the level of consciousness. It builds on the Integrated Information Theory (IIT) prediction that consciousness requires both *integration* (widespread causal interactions) and *differentiation* (a large repertoire of distinct states) within the thalamocortical system.

PCI operationalises this by combining two ingredients:

1. **Perturbation** — A transcranial magnetic stimulation (TMS) pulse directly activates a cortical region, triggering a cascade of evoked activity recorded by high-density EEG.
2. **Complexity measurement** — The spatiotemporal pattern of significant cortical responses is binarised, then compressed using the Lempel–Ziv algorithm (LZ76). The normalised compression ratio yields a single number, $\text{PCI} \in [0, 1]$.

Casali et al. demonstrated that PCI reliably discriminates conscious from unconscious brain states across diverse conditions: wakefulness, REM and NREM sleep, anaesthesia (midazolam, propofol, xenon), locked-in syndrome, vegetative state (VS/UWS), and minimally conscious state (MCS). A threshold of $\text{PCI}^* = 0.31$ was never exceeded during confirmed unconsciousness.

This software implements the full PCI pipeline in Python using MNE-Python for EEG processing, NumPy for numerical computation, and Streamlit for the interactive web interface.

2 Mathematical Formulation

2.1 Source Entropy

Given a binary significance matrix $\mathbf{SS}(x, t) \in \{0, 1\}^{L_1 \times L_2}$ where L_1 is the number of sources and L_2 the number of post-stimulus time samples, define:

$$p_1 = \frac{1}{L} \sum_{x=1}^{L_1} \sum_{t=1}^{L_2} \mathbf{SS}(x, t), \quad L = L_1 \cdot L_2 \quad (1)$$

The binary source entropy is:

$$H(L) = -p_1 \log_2 p_1 - (1 - p_1) \log_2 (1 - p_1) \quad (2)$$

2.2 Lempel–Ziv Complexity (LZ76)

The matrix $\mathbf{SS}$ is first sorted by descending row activation (optimal ordination, Section 4), then flattened in column-major (Fortran) order into a binary string $s = s_1 s_2 \cdots s_L$.

The LZ76 exhaustive-parsing algorithm [5, 6] processes s as follows:

Algorithm: LZ76 Exhaustive Parsing

1. Initialise word count $c \leftarrow 1$, scan position $i \leftarrow 1$
2. While $i < L$:
 - a. Find the longest prefix $s[i:i+k]$ that appears as a substring in the history $s[0:i]$
 - b. Set $i \leftarrow i + k + 1$ (new word = match of length $k + 1$ novel symbol)
 - c. Increment $c \leftarrow c + 1$
3. Return $c_L = c$

2.3 PCI Normalisation

The Perturbational Complexity Index is defined as:

$$\text{PCI} = \frac{c_L \cdot \log_2 L}{L \cdot H(L)} \quad (3)$$

The denominator $L \cdot H(L)/\log_2 L$ represents the asymptotic Lempel–Ziv complexity of a random binary sequence of length L with the same proportion of ones p_1 (Lempel & Ziv, 1976). Thus $\text{PCI} \approx 1$ for maximally random (complex) patterns and $\text{PCI} \ll 1$ for simple, repetitive patterns.

2.4 Temporal Variant PCI^T

The transposed matrix $\mathbf{SS}^T$ is flattened in column-major order (scanning source-by-source rather than time-by-time) and compressed with the same LZ76 algorithm to yield c_L^T . The temporal PCI is normalised identically:

$$\text{PCI}^T = \frac{c_L^T \cdot \log_2 L}{L \cdot H(L)} \quad (4)$$

2.5 Temporal Evolution $\text{PCI}(t)$

The cumulative PCI at each time step $t = 1, \dots, L_2$ is computed as:

$$\text{PCI}(t) = \frac{c_i(t) \cdot \log_2 \ell(t)}{\ell(t) \cdot H(L)} \quad (5)$$

where $\ell(t) = L_1 \cdot t$ and $c_i(t) = \text{LZ76}(\mathbf{SS}_{\text{sorted}}[:, 1:t])$. Note that $H(L)$ is the entropy of the *full* matrix, not of the sub-matrix up to time t .

3 Bootstrap Significance Testing

3.1 Procedure

The binary significance matrix $\mathbf{SS}(x, t)$ is constructed via non-parametric bootstrap with Global Maximum Statistics (GMS) correction [4]:

1. **Trial averaging:** Compute the evoked response $\bar{d}(x, t) = \frac{1}{N} \sum_{n=1}^N d_n(x, t)$, where N is the number of accepted epochs.
2. **Z-scoring:** For each source x , compute baseline statistics $\mu_x = \text{mean}(\bar{d}(x, t_{\text{base}}))$ and $\sigma_x = \text{std}(\bar{d}(x, t_{\text{base}}), \text{ddof}=1)$, then:

$$z(x, t) = \frac{\bar{d}(x, t) - \mu_x}{\sigma_x} \quad (6)$$

3. **Dead channel exclusion:** Sources with $\sigma_x < 10^{-12}$ are excluded from the bootstrap and assigned all-zero rows in $\mathbf{SS}$.
4. **Bootstrap null distribution:** For $b = 1, \dots, B$ (default $B = 500$):
 - a. Draw n_{post} random indices $\mathbf{j} = (j_1, \dots, j_{n_{\text{post}}})$ from baseline time points — **the same indices for all sources** (correlated resampling).
 - b. Compute $m_b = \max_{x,k} |z_{\text{base}}(x, j_k)|$ — the global maximum across all alive sources and resampled time points.

5. **Threshold:** $\tau = P_{1-\alpha}(\{m_b\})$, i.e. the $(1-\alpha)$ -th percentile of null maxima (default $\alpha = 0.01$).

6. **Binarisation:**

$$\mathbf{SS}(x, t) = \begin{cases} 1 & \text{if } |z(x, t)| > \tau \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

3.2 Correlated vs. Independent Resampling

Critical Fix: Correlated Bootstrap

The original code drew *independent* random time indices per source: `idx = rng.randint(0, n_base, size=(n_src, n_post))`. This broke the spatial correlation structure between sources, producing a less conservative (lower) threshold τ and systematically more false positives in **SS**.

The corrected version draws a *single* set of time indices shared across all sources: `idx = rng.randint(0, n_base, size=n_post)`, preserving the natural cross-source dependence and matching the Global Maximum Statistics framework [4].

4 Source Sorting (Optimal Ordination)

Before Lempel–Ziv compression, sources (rows of **SS**) are sorted in descending order of total activation count $a_x = \sum_t \mathbf{SS}(x, t)$ to minimise artefactual spatial complexity (Supplementary Material §3.3 of [1]).

4.1 Deterministic Tie-Breaking

When multiple sources share the same activation count a_x , a secondary key is needed to ensure the sorted order is independent of the original row permutation. We interpret each row as a binary number:

$$v_x = \sum_{t=1}^{L_2} \mathbf{SS}(x, t) \cdot 2^{L_2-t} \quad (8)$$

The final sort uses the composite key $(-a_x, -v_x)$ via NumPy’s `lexsort`, yielding a fully deterministic, permutation-invariant ordering. For matrices wider than 1020 columns (where 2^{L_2} would overflow `float64`), a fallback to Python tuple comparison is used.

5 Preprocessing Pipeline

The preprocessing follows Casali et al. (2013) with the following steps:

1. **TMS artifact interpolation:** Linear interpolation over $[-2, +10]$ ms around each TMS pulse onset. A new validation function warns if the interpolation window overlaps the post-stimulus analysis window.
2. **Bandpass filtering:** 0.1–45 Hz zero-phase FIR filter (MNE default).
3. **Resampling:** Downsample to 362.5 Hz if original $f_s > 400$ Hz, matching the temporal resolution in the original publication.
4. **Epoching:** Segments of $[-500, +350]$ ms around each TMS pulse are extracted.
5. **Artifact rejection:** Epochs with peak-to-peak amplitude $> 150 \mu\text{V}$ in any channel are rejected.

6. **Source estimation:** Either Current Source Density (CSD) via the surface Laplacian or direct sensor-level analysis. See Section 5.1 for caveats.
7. **SNR computation:** $\text{SNR} = |\bar{A}|_{\text{post}}/\bar{\sigma}_{\text{base}}$. Sessions with $\text{SNR} < 1.4$ were excluded in the original study.

5.1 CSD vs. Inverse Solution

Important Caveat

Casali et al. (2013) used a 3-sphere BERG forward model with ~ 5000 cortical dipoles as an inverse solution for source estimation. Our implementation uses **Current Source Density (CSD)**, which is a spatial filter (surface Laplacian) producing one “source” per EEG channel. The published thresholds $\text{PCI}^* = 0.31$ (unconscious ceiling) and $\text{PCI} \geq 0.44$ (conscious range) were **not validated with CSD**. Casarotto et al. [3] showed that sensor-level PCI retains discriminative power, but thresholds may require recalibration.

6 Istisna Mode: Exception Handling for Mislabelled Triggers

6.1 Motivation

In some TMS-EEG recording setups, hardware TTL pulses from the TMS stimulator are wired to the “Response” port of the EEG amplifier rather than the “Stimulus” port. This results in TMS onset markers being saved with response-type labels (e.g., “R 1”, “Response/R 128”) in BrainVision files.

The *istisna* (Arabic: exception) mode detects this mislabelling by analysing the temporal regularity of response-labelled event trains.

6.2 Detection Algorithm

Given an event train $\{t_1, t_2, \dots, t_N\}$ labelled as “response”:

1. Require $N \geq 30$ (minimum total events).
2. Split into temporal blocks by gaps exceeding 12 s.
3. For each block with ≥ 10 events, compute inter-stimulus intervals $\Delta t_i = (t_{i+1} - t_i)/f_s$ and evaluate:
 - Median $\text{ISI} \in [0.2, 10.0]$ s (typical TMS stimulation rates)
 - $\text{CV}(\text{ISI}) = \sigma_{\Delta t}/\mu_{\Delta t} \leq 0.05$ (5% jitter tolerance)
4. If any block satisfies both criteria $\Rightarrow \text{is_stimulation_like} = \text{True}$.

6.3 User Interaction

When periodicity is detected, the user is presented with a checkbox to enable *istisna* mode. If enabled, response markers are reclassified as TMS triggers. Per-block selection is available for multi-site or multi-condition recordings where only some blocks represent TMS.

7 Quality Assessment

The software performs automated quality checks against physiological ranges from the literature:

Table 1: Quality assessment criteria and expected ranges.

Parameter	PASS Range	WARN Range	FAIL Condition
Active %	[4, 15]	$(2, 4) \cup (15, 25)$	< 2 or > 25
Entropy H	[0.15, 0.60]	edge values	< 0.05 or > 0.70
Threshold τ	[3.0, 6.0]	(2.5, 3.0)	> 8.0
SNR	≥ 1.4	[1.0, 1.4)	< 1.0
PCI	≤ 1.10	—	> 1.10

8 Clinical Interpretation

Table 2: PCI interpretation thresholds from Casali et al. (2013).

PCI Range	Interpretation	Conditions
$\text{PCI} \geq 0.44$	Conscious	Wakefulness, REM sleep, locked-in
$0.31 \leq \text{PCI} < 0.44$	Intermediate (MCS)	Minimally conscious state
$\text{PCI} < 0.31$	Unconscious	NREM sleep, anaesthesia, VS/UWS

Note: These thresholds were derived using full inverse-solution source reconstruction. When using CSD or sensor-level analysis (as in this implementation), the absolute PCI values may differ and the thresholds should be interpreted with caution.

9 Algorithm Flowchart

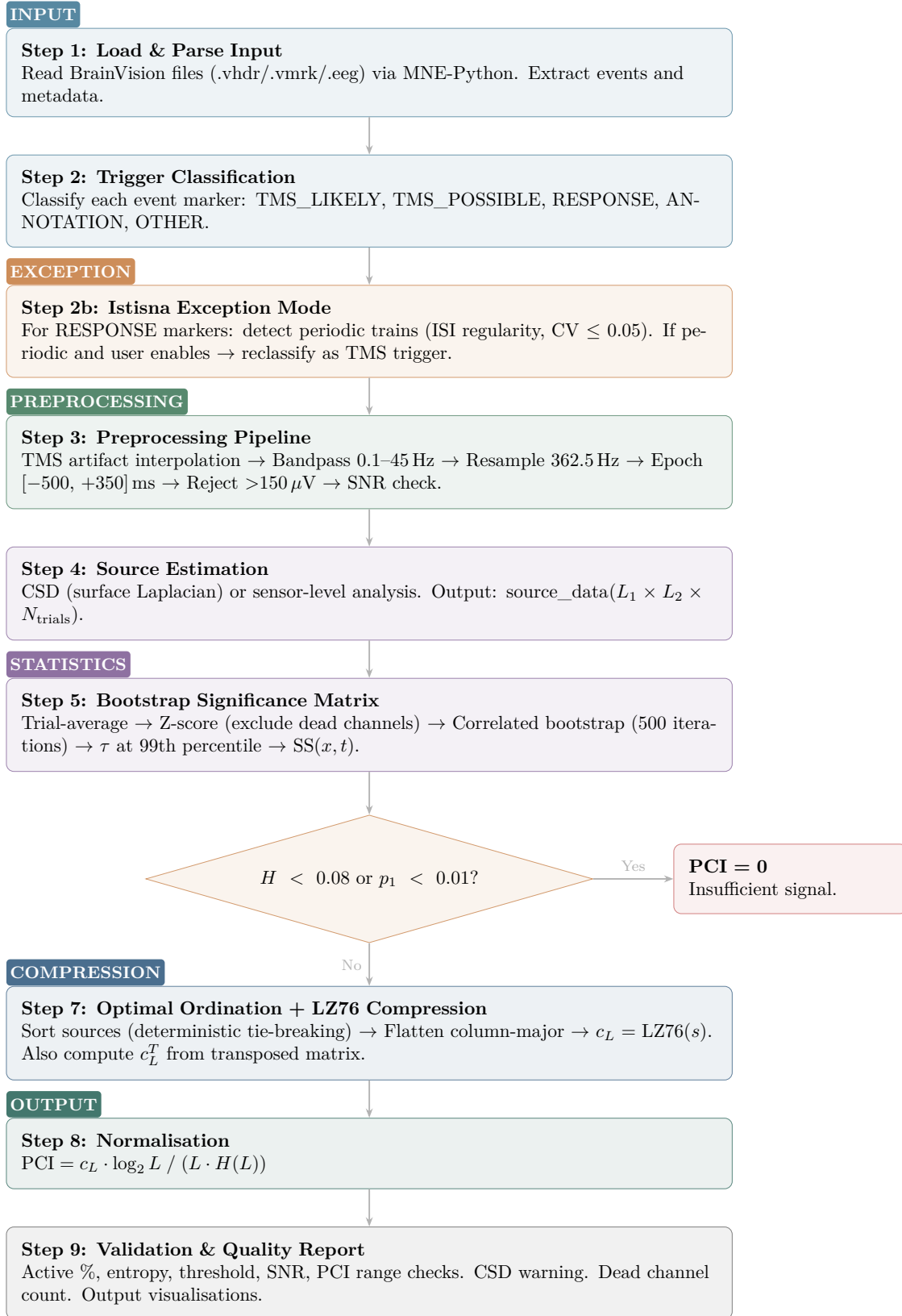


Figure 1: Complete PCI algorithm flowchart showing the processing pipeline from raw EEG data to the final PCI value with quality assessment.

10 Corrections and Improvements

The following issues were identified during a thorough code review against Casali et al. (2013) and resolved:

10.1 Critical Fixes

1. **Bootstrap resampling (correlated):** The bootstrap previously drew independent random time indices per source, breaking spatial correlations. Fixed to draw a single set of indices shared across all sources, matching GMS [4].
2. **Dead channel exclusion:** Sources with $\sigma \approx 0$ were previously floored to the 5th percentile of non-zero sigmas, inflating their z-scores. Now these channels are excluded entirely and assigned all-zero rows in **SS**.
3. **Permutation-invariant source sorting:** Added deterministic tie-breaking using row content as a binary number (Eq. 8), ensuring identical results regardless of the original row order.

10.2 Moderate Fixes

1. **Window overlap validation:** New warning when the TMS artifact interpolation window extends beyond the post-stimulus analysis start.
2. **Threshold harmonisation:** `compute_pci_temporal` now uses the same quality gates (`min_entropy = 0.08`, `min_p1 = 0.01`) as `compute_pci`.
3. **CSD documentation:** Prominent warnings added that CSD is not equivalent to full inverse-solution source reconstruction and that published thresholds may not apply.
4. **Istisna CV threshold:** Raised from 0.02 to 0.05 to accommodate jitter from manual TMS triggering and low sampling rates.
5. **Configurable random seed:** The bootstrap accepts a `seed` parameter (default 42, `None` for non-deterministic).

11 Software Architecture

Table 3: Project file structure and responsibilities.

File	Lines	Responsibility
<code>pci.py</code>	~950	Core algorithm: entropy, LZ76, bootstrap, PCI, trigger classification
<code>app.py</code>	~1230	Streamlit UI: file upload, parameter controls, visualisations
<code>test_pci.py</code>	~370	41 unit tests covering all algorithm components

Dependencies: Python 3.8+, NumPy, MNE-Python, Streamlit, Matplotlib, pytest.

12 Test Suite

The test suite (`tests/test_pci.py`) contains 41 tests organised into 9 classes:

Table 4: Test suite coverage summary.

Test Class	Tests	Count
TestLZComplexity	LZ76 correctness on known sequences	7
TestEntropy	Binary entropy edge cases and known values	5
TestPCICore	PCI formula, zero matrices, simple patterns	5
TestTriggerClassification	Keyword matching, count ranges, edge cases	7
TestResponseTrainDetection	Periodicity detection, block splitting	5
TestValidation	Quality checks, trigger diagnostics	3
TestBootstrapSignificance	Shape, correlated resampling, dead channels, seed	5
TestWindowOverlap	Overlap warning, no-overlap silence	2
TestPCITemporal	Low entropy returns zeros, convergence	2
Total		41

All 41 tests pass as of the final verification (February 2026).

13 Future Work

- **PCI_{ST}** (Comolatti et al., 2019 [2]): Implement the state-transition variant using SVD dimensionality reduction and recurrence quantification, which computes faster and has been validated on larger cohorts.
- **Bootstrap confidence intervals**: Report PCI uncertainty ranges via jackknife or bootstrap-of-bootstrap procedures.
- **ISI histogram**: Visual verification tool for istisna mode.
- **Additional file formats**: Support for EDF/BDF/EEGLAB beyond BrainVision.
- **Full inverse solution**: Integrate MNE’s minimum-norm estimation for closer alignment with the original Casali et al. methodology.

References

- [1] Casali AG, Gosseries O, Rosanova M, et al. A theoretically based index of consciousness independent of sensory processing and behavior. *Science Translational Medicine*, 5(198):198ra105, 2013.
- [2] Comolatti R, Pigorini A, Casarotto S, et al. A fast and general method to empirically estimate the complexity of brain responses to transcranial and intracranial stimulations. *Brain Stimulation*, 12(5):1280–1289, 2019.
- [3] Casarotto S, Comanducci A, Rosanova M, et al. Stratification of unresponsive patients by an independently validated index of brain complexity. *Annals of Neurology*, 80(5):718–729, 2016.
- [4] Nichols TE, Holmes AP. Nonparametric permutation tests for functional neuroimaging: A primer with examples. *Human Brain Mapping*, 15(1):1–25, 2002.
- [5] Lempel A, Ziv J. On the complexity of finite sequences. *IEEE Transactions on Information Theory*, 22(1):75–81, 1976.
- [6] Kaspar F, Schuster HG. Easily calculable measure for the complexity of spatiotemporal patterns. *Physical Review A*, 36(2):842–848, 1987.

- [7] Rogasch NC, Sullivan C, Thomson RH, et al. Analysing concurrent transcranial magnetic stimulation and electroencephalographic data: A review and introduction to the open-source TESA software. *NeuroImage*, 147:934–951, 2017.